Why `distribution_sort` Runs in Expected Linear Time

Analysis of `Linear Time Sort/distribution_sort.py`

October 2026

Abstract

`distribution_sort` fits a model $\widehat{F}$ of the cumulative distribution function (CDF) of its input, drops each element x into bucket $\left\lfloor n\,\widehat{F}(x)\right\rfloor$, and sorts each bucket. We prove that it always returns the input in sorted order; that on every input it takes $O(n \log n)$ time; and that when the values are independent draws from a continuous distribution, its expected time is $(34 + 5\chi^2)\,n + O(\sqrt{n})$, where χ^2 is the χ^2-distance between the data's density and the model's density. In particular the expected time is linear whenever the model fits, and even when it fits only roughly. We also prove the limit of the method: if one value repeats a constant fraction of the time, no CDF model can avoid $\Omega(n \log n)$ time, which is what the benchmark on file timestamps showed.

Results at a glance

Situation	Bound	Where proved
Any input	always sorted	Theorem 3.2
Any input	$\le 5n \log_2 n + 98n + C$ steps	Theorem 5.1
Model exactly right (idealised)	$\le 34n + C$ expected	Corollary 6.2
Independent continuous data, model at χ^2-distance χ^2	$\le (34 + 5\chi^2)n + O(\sqrt{n})$ expected	Theorem 6.8
Uniform data, uniform model chosen	$\le 34n + O(\sqrt{n})$ expected	Corollary 6.10
Any density with $\mu \le f \le M$ on a bounded interval, empirical model	$\le (29 + 15M/\mu)\,n + O(\sqrt{n})$ expected	Corollary 6.11
A value with probability $\pi > 0$	$\ge \frac{\pi n}{2} \log_2 \frac{\pi n}{2}$ expected, for any model	Proposition 7.1

Here C is a constant that does not depend on n (it covers the work done on the fixed-size sample). The constants are upper bounds chosen to make each step easy to check, not predictions of the measured running time.

The idea in one paragraph. Placing an element costs $O(1)$, so placement is linear. Sorting a bucket of k elements costs at most a constant times k^2, so all the bucket sorts together cost $O\big(\sum_j N_j^2\big)$, where N_j is the size of bucket j. If the model were the true CDF F, then $F(X)$ would be uniform on $[0, 1]$ (Lemma 6.1), so each element would land in each bucket with probability exactly $1/n$. Each N_j would then be Binomial$(n, 1/n)$, with $\mathbb{E}[N_j^2] < 2$, and the bucket sorts would cost less than a constant times $2n$ in expectation. The rest of the proof removes two idealisations:

the model is only approximately F (this costs a factor $1+\chi^2$), and it is built from the same data it then sorts (this costs only $O(\sqrt{n})$).

Relation to the earlier proof. `distribution_sort_proof.tex` analyses an earlier version of the algorithm, which has recursion, discrete models and an all-equal check. This document follows the same structure for the current, simpler version. One thing is new here: the earlier proof treated the model as independent of the data (its assumption (A2)), whereas Lemma 6.3 below proves the expected-time bound without that assumption.

1 The algorithm

The code has two constants: `BIG_BUCKET` $= 64$ and `SAMPLE_SIZE` $= 1000$. A call on a list of n numbers performs these steps.

D1. Small input. If $n \le 64$, insertion-sort a copy of the list and return it.

D2. Sample. Let $s = \min(n, 1000)$. Choose s distinct positions uniformly at random with `random.sample`, and add the list's minimum $x_{\min}$ and maximum $x_{\max}$. Sort the resulting $s+2$ values into $q_0 \le q_1 \le \cdots \le q_L$, where $L = s+1$. Note $q_0 = x_{\min}$ and $q_L = x_{\max}$.

D3. Model. Fit three candidates to $q_0, \ldots, q_L$: a uniform CDF on $[q_0, q_L]$, a Normal CDF with the sample mean and standard deviation, and an Exponential CDF shifted to start at q_0. A candidate is accepted if its Kolmogorov–Smirnov distance to the sample is below $1.36/\sqrt{L+1}$; the accepted candidate with the smallest distance becomes the model $\widehat{F}$. If none is accepted, $\widehat{F}$ is the *empirical CDF*: with $c(x) = \#\{i : q_i \le x\}$,

$$\widehat{F}(x) = \begin{cases} 0 & c(x) = 0, \\ \dfrac{1}{L}\Big(c(x) - 1 + \dfrac{x - q_{c(x)-1}}{q_{c(x)} - q_{c(x)-1}}\Big) & 1 \le c(x) \le L, \\ 1 & c(x) = L+1. \end{cases}$$

(When $1 \le c(x) \le L$ we have $q_{c(x)-1} \le x < q_{c(x)}$, so the denominator is positive.)

D4. Place. Create n empty buckets and append each element x to bucket

$$\beta(x) = \min\big(n-1,\ \max(0,\ \mathrm{int}(n\,\widehat{F}(x)))\big).$$

D5. Finish each bucket. A bucket of size $k \le 64$ is insertion-sorted. A bucket of size $k > 64$ is merge-sorted (the *safety net*).

D6. Join the buckets in order and return the result.

Conventions. We use the RAM model. A *step* is one iteration of any loop, including loops inside library routines (copying or slicing a list, `min`, `max`, `sorted`, `bisect`, `random.sample`). The straight-line code around an iteration counts as part of it; a call to `erf` or `exp` counts as one step; appending to a list counts as one step (CPython over-allocates lists, so this holds in amortised time up to a constant factor). Each step costs $O(1)$ machine time. Real numbers are treated as exact (Remark 8.3 discusses floating point). All logarithms are base 2. $T(n)$ is the number of steps of a call on n elements.

2 Elementary facts

Fact 2.1 (Insertion sort)**.** *Insertion sort on $k \geq 1$ elements takes at most $(k-1) + \binom{k}{2} \leq k^2$ steps.*

Proof. The outer loop runs $k-1$ times. Each inner-loop step moves one element past the element being inserted, and each of the $\binom{k}{2}$ pairs is moved past each other at most once. Finally $(k-1) + \frac{k(k-1)}{2} \leq k^2$. □

Fact 2.2 (Merge sort)**.** `merge_sort` *on $k \geq 1$ elements takes at most $5k(\lceil \log k \rceil + 1)$ steps and at least $k(\lfloor \log k \rfloor + 1)$ steps.*

Proof. A call on $k \geq 2$ elements copies its input (`list(values)`), slices it into halves of sizes $\lfloor k/2 \rfloor$ and $\lceil k/2 \rceil$, recurses on both, and merges. Using $\lfloor \lfloor x \rfloor /2 \rfloor = \lfloor x/2 \rfloor$ and $\lceil \lceil x \rceil /2 \rceil = \lceil x/2 \rceil$, every call at recursion depth d has between $\lfloor k/2^d \rfloor$ and $\lceil k/2^d \rceil$ elements.

Upper bound. At depth $\lceil \log k \rceil$ every call has one element, so there are at most $\lceil \log k \rceil + 1$ depths. The calls at one depth work on disjoint parts of the input. Per element at one depth, a call spends at most 5 steps: one to copy it, one to slice it, at most two in merging (one loop iteration, or one slice plus one extend for leftovers), and one for its share of call overhead (a depth has at most as many calls as elements).

Lower bound. For $d \leq \lfloor \log k \rfloor$ every call at depth d has at least $\lfloor k/2^d \rfloor \geq 1$ element, and every call at depth $d-1$ has at least 2 elements and therefore recurses. So every element is in some call at each depth $0, 1, \ldots, \lfloor \log k \rfloor$, and each such call copies it once. □

Fact 2.3 (Binary search)**.** *Evaluating the empirical CDF uses* `bisect_right` *on $L + 1 \leq 1002$ points, which takes at most 10 iterations.*

Proof. The search interval starts with length $L + 1 \leq 1002$ and each iteration at least halves it (rounding down), so there are at most $\lfloor \log 1002 \rfloor + 1 = 10$ iterations. □

Fact 2.4 (`random.sample`)**.** *Drawing $s \leq n$ positions with* `random.sample` *takes at most $5n$ steps in expectation, and the number of steps is independent of which positions are returned.*

Proof. CPython computes $\sigma = 21$, plus $4^{\lceil \log_4(3s) \rceil} \geq 3s$ if $s > 5$. Each random draw calls `randbelow`, a rejection loop that takes fewer than 2 iterations on average.

- If $n \leq \sigma$, it allocates the result (s steps), copies the list (n steps) and makes s draws and swaps (fewer than $3s$ steps on average), at most $n + 4s \leq 5n$ in total.
- Otherwise $n > \sigma \geq 3s$, and it draws positions until it has s distinct ones. While fewer than s have been chosen, a draw repeats an earlier one with probability below $s/n < 1/3$, so each new position needs fewer than 1.5 draws on average. The total is at most $s + 1.5s \cdot 3 + s < 7s < 3n$ on average.

In both cases the only randomness in the step count comes from rejection loops, and in a rejection loop the number of trials is independent of the value finally accepted. So the step count is independent of the positions returned. □

Fact 2.5 (Second moment of a binomial)**.** *If $N \sim \text{Binomial}(n, p)$, then $\mathbb{E}[N^2] = np(1-p) + n^2p^2 \leq np + n^2p^2$.*

Fact 2.6 (Cauchy–Schwarz inequality)**.** *For non-negative functions u, v on an interval I, $\int_I uv \leq \left(\int_I u^2\right)^{1/2} \left(\int_I v^2\right)^{1/2}$.*

3 Correctness

Lemma 3.1 (The model never decreases)**.** *Every model built in step D3 is non-decreasing on* $[x_{\min}, x_{\max}]$*, which contains every input value. Hence the bucket map is non-decreasing: if* $x \le y$ *then* $\beta(x) \le \beta(y)$*.*

Proof. The Normal model is a CDF. The uniform model $(x - q_0)/(q_L - q_0)$ is increasing. The Exponential model is 0 up to q_0 and $1 - e^{-(x-q_0)/\lambda}$ after it, with $\lambda > 0$; both pieces are non-decreasing and they agree at q_0.

For the empirical CDF, $c(x)$ is non-decreasing in x. While $c(x)$ stays equal to some $c \in [1, L]$, the formula is linear with positive slope and takes values in $[\frac{c-1}{L}, \frac{c}{L})$. When $c(x)$ jumps from c to $c' > c$ (at a sample point, possibly repeated), the value just before the jump is below $\frac{c}{L}$ and the value at the jump is $\frac{c'-1}{L} \ge \frac{c}{L}$, or 1 if $c' = L + 1$. Below q_0 the value is 0, the smallest possible.

Finally, β composes $\widehat{F}$ with three non-decreasing maps: multiplying by n, `int`, and clamping to $[0, n-1]$. (`int` truncates towards zero rather than rounding down, but the two differ only on negative numbers, which the clamp sends to 0 either way.) □

Theorem 3.2 (Correctness)**.** *For every input list,* `distribution_sort` *stops and returns its elements in non-decreasing order.*

Proof. If $n \le 64$, insertion sort is correct. Otherwise, let x be in bucket j and y in bucket j' with $j < j'$. If $x > y$, Lemma 3.1 would give $j = \beta(x) \ge \beta(y) = j'$, a contradiction. So every element of an earlier bucket is at most every element of a later bucket. Each bucket is sorted by insertion sort or merge sort, both of which are correct and stop. Joining sorted buckets that are in order gives a sorted list. □

4 Where the time goes

Lemma 4.1 (Work outside the bucket sorts)**.** *For* $n > 64$*, the work outside the bucket sorts in step D5 is* $W \le A\,n + C$ *in expectation, with* $A = 24$ *and* C *a constant that does not depend on* n*. The bound also holds conditional on the positions sampled and on the data.*

Proof. The table lists every part of steps D1–D6 except the bucket sorts.

Work	Steps ≤	Reason
D1 copy the input	n	one pass
D2 `random.sample`	$5n$	Fact 2.4, in expectation
D2 min and max	$2n$	one pass each
D2 build the sample list	$n + 2$	$s + 2 \le n + 2$ elements
D2 sort the sample; D3 fit three models, KS distances, build $\widehat{F}$	C	every loop runs over the $L + 1 \le 1002$ sample points, so the work is bounded by a constant
D4 create n buckets	n	one pass
D4 place each element	$12n$	per element: 1 iteration, ≤ 10 search steps (Fact 2.3), 1 append
D5, D6 visit buckets and join	$2n$	one iteration per bucket, one copy per element
Total	$24n + C$	

The constant 2 from the sample list is absorbed into C. Only the `random.sample` row is random, and by Fact 2.4 its step count is independent of the positions returned and of the data (which the sampler never looks at), so conditioning on them does not change its expectation. □

Lemma 4.2 (Cost of one bucket)**.** *Let* $\mathrm{cost}(k)$ *be the steps used to sort a bucket of size* k *in step D5. Then*

$$\mathrm{cost}(k) \le \begin{cases} k^2 \le 64k & k \le 64, \\ 5k(\lceil \log k \rceil + 1) & k > 64, \end{cases} \qquad \textit{and in all cases} \qquad \mathrm{cost}(k) \le 5k^2.$$

Proof. The first case is Fact 2.1 and the second is Fact 2.2. Since $\lceil \log k \rceil + 1 \le k$ for every $k \ge 1$, both cases are at most $5k^2$. □

Writing N_j for the size of bucket j, the two lemmas give, for $n > 64$,

$$T(n) = W + \sum_{j=0}^{n-1} \mathrm{cost}(N_j), \qquad \mathbb{E}[W] \le An + C, \qquad \sum_j N_j = n. \tag{1}$$

5 Worst case: $O(n \log n)$ for every input

Theorem 5.1 (Worst case)**.** *For every input with* $n > 64$ *elements,*

$$\mathbb{E}[T(n)] \le 5n \log n + (A + 74)\, n + C = 5n \log n + 98n + C,$$

where the expectation is only over `random.sample`*'s internal draws. For* $n \le 64$*,* $T(n) \le n + n^2 \le 65n$*.*

Proof. By Lemma 4.2, a bucket with $N_j \le 64$ costs at most $64N_j$, and a bucket with $N_j > 64$ costs at most $5N_j(\lceil \log N_j \rceil + 1) \le 5N_j(\log n + 2)$, since $N_j \le n$. Adding over all buckets and using $\sum_j N_j = n$,

$$\sum_j \mathrm{cost}(N_j) \le 64n + 5n(\log n + 2) = 5n \log n + 74n.$$

Add $\mathbb{E}[W] \le An + C$ from (1). For $n \le 64$, step D1 copies the list and insertion-sorts it (Fact 2.1). □

Remark 5.2. This bound is what the safety net buys. Without it, one overfull bucket of size $\Theta(n)$ would be insertion-sorted in $\Theta(n^2)$ time. Replacing `random.sample` by a partial Fisher–Yates shuffle would make the bound hold on every run, without the expectation.

6 Expected linear time for continuous data

6.1 The ideal case

Lemma 6.1 (Probability integral transform)**.** *If* X *has a continuous CDF* F*, then* $F(X)$ *is uniform on* $[0, 1]$*.*

Proof. Fix $u \in (0, 1)$. Since F is continuous and non-decreasing, with limits 0 and 1, the set $\{x : F(x) \le u\}$ is non-empty and bounded above, and it contains its supremum x_u; continuity also gives $F(x_u) = u$. Then $F(X) \le u$ exactly when $X \le x_u$, so $\mathbb{P}(F(X) \le u) = F(x_u) = u$. □

Corollary 6.2 (Ideal model)**.** *Suppose the data are independent draws from a continuous CDF F and, hypothetically, $\widehat{F} = F$ and $\widehat{F}$ were chosen independently of the data. Then $\mathbb{E}[T(n)] \le (A+10)\,n + C = 34n + C$.*

Proof. By Lemma 6.1, element i lands in bucket j when $nF(X_i)$ lies in $[j, j+1)$ (or in $[n-1, n]$ for the last bucket), which has probability exactly $1/n$. The elements are independent, so $N_j \sim \text{Binomial}(n, 1/n)$ and, by Fact 2.5, $\mathbb{E}[N_j^2] = 1 - \frac{1}{n} + 1 < 2$. By Lemma 4.2 and (1), $\mathbb{E}[T] \le An + C + 5\sum_j \mathbb{E}[N_j^2] < An + C + 10n$. □

The code differs from this ideal in two ways: $\widehat{F}$ is only an approximation of F, and $\widehat{F}$ is computed from a sample of the same list it then sorts. The next two subsections deal with each.

6.2 The model comes from the data

Assumption (A). The input values $X_1, \dots, X_n$ are independent and identically distributed with a density f and CDF F. (So, with probability 1, no two values are equal.)

Notation. Let J be the set of sampled positions, ι and κ the positions of the minimum $x_{\min}$ and maximum $x_{\max}$, and $E = J \cup \{\iota, \kappa\}$, so $|E| \le s+2$. Let R be the remaining positions. Let $\mathcal{G}$ denote the information $(J, \iota, \kappa, (X_i)_{i\in E})$. Write $Z = F(x_{\max}) - F(x_{\min})$ and let

$$f_*(x) = \frac{f(x)}{Z} \quad \text{for } x_{\min} < x < x_{\max}, \qquad f_*(x) = 0 \text{ otherwise}$$

be the density f restricted to $(x_{\min}, x_{\max})$.

Lemma 6.3 (Conditioning on the model's inputs)**.** (i) *The model $\widehat{F}$, and the bucket of every element in E, are determined by $\mathcal{G}$.* (ii) *Given $\mathcal{G}$, the values $(X_i)_{i\in R}$ are independent with density f_*.*

Proof. (i) Steps D2 and D3 use only the sampled values, $x_{\min} = X_\iota$ and $x_{\max} = X_\kappa$.

(ii) `random.sample` uses its own random numbers, so J is independent of the data. Fix J, $\iota = i_0$ and $\kappa = k_0$ (with $i_0 \ne k_0$, which holds with probability 1 when $n \ge 2$). The joint density of the data is $\prod_i f(x_i)$, restricted to the event that $x_{i_0} < x_i < x_{k_0}$ for every $i \ne i_0, k_0$. Now also fix the values x_i for $i \in E$. The conditions involving positions in E are then fixed, and what remains of the density of $(x_i)_{i\in R}$ is proportional to $\prod_{i\in R} f(x_i)\,\mathbf{1}\{x_{\min} < x_i < x_{\max}\}$. This is a product, so the values are independent, each with density proportional to $f \cdot \mathbf{1}_{(x_{\min}, x_{\max})}$, which is f_*. □

6.3 The model is only approximately right

From here on, condition on $\mathcal{G}$, so that $\widehat{F}$ is a fixed function. Every model in D3 is continuous with a derivative $\widehat{f}$ except at finitely many points, apart from upward jumps of the empirical CDF at repeated sample points (which happen with probability 0 under Assumption (A)).

Definition 6.4. The χ^2*-distance* from the data to the model is

$$\chi^2 = \chi^2(f_* \,\|\, \widehat{f}) = \int_{x_{\min}}^{x_{\max}} \frac{f_*(x)^2}{\widehat{f}(x)}\,dx - 1,$$

taken to be $+\infty$ if $\widehat{f} = 0$ on a set where $f_* > 0$. By Fact 2.6, $1 = \int f_* \le (\int f_*^2/\widehat{f})^{1/2}(\int \widehat{f})^{1/2} \le (\int f_*^2/\widehat{f})^{1/2}$, so $\chi^2 \ge 0$, with equality exactly when $\widehat{f} = f_*$. For each bucket j let $I_j = \{x \in (x_{\min}, x_{\max}) : \beta(x) = j\}$ and $p_j = \int_{I_j} f_*$.

Lemma 6.5 (The model gives each bucket mass at most $1/n$)**.** *For every bucket j, $\int_{I_j} \widehat{f} \le 1/n$.*

Proof. By Lemma 3.1, I_j is an interval. On I_j, $n\widehat{F}$ lies in $[j, j+1)$, or in $[0,1)$ for $j = 0$, or in $[n-1, n]$ for $j = n-1$ (because of the clamp, using $0 \le \widehat{F} \le 1$ on $[x_{\min}, x_{\max}]$). So $\widehat{F}$ increases by at most $1/n$ across I_j. Split I_j at the finitely many points where $\widehat{F}$ is not differentiable. On each piece, $\int \widehat{f}$ equals the increase of $\widehat{F}$ across the piece, and jumps at the split points only add to the total increase. So $\int_{I_j} \widehat{f} \le 1/n$. □

Lemma 6.6 (Collisions are rare)**.** $\displaystyle\sum_j p_j^2 \le \frac{1+\chi^2}{n}$.

Proof. If $\chi^2 = \infty$ there is nothing to prove. Otherwise, by Fact 2.6 and Lemma 6.5,

$$p_j = \int_{I_j} \frac{f_*}{\sqrt{\widehat{f}}} \cdot \sqrt{\widehat{f}} \le \Big(\int_{I_j} \frac{f_*^2}{\widehat{f}}\Big)^{1/2} \Big(\int_{I_j} \widehat{f}\Big)^{1/2} \le \Big(\frac{1}{n} \int_{I_j} \frac{f_*^2}{\widehat{f}}\Big)^{1/2}.$$

Square, and add over j; the I_j do not overlap and cover $(x_{\min}, x_{\max})$, so $\sum_j p_j^2 \le \frac{1}{n} \int f_*^2/\widehat{f} = \frac{1+\chi^2}{n}$. □

Lemma 6.7 (Bucket sizes)**.** $\mathbb{E}\Big[\displaystyle\sum_j N_j^2 \,\Big|\, \mathcal{G}\Big] \le (2+\chi^2)\, n + 2(s+2)\sqrt{(1+\chi^2)\, n} + (s+2)^2$.

Proof. Split $N_j = N_j' + S_j$, where N_j' counts elements of R in bucket j and S_j counts elements of E. By Lemma 6.3, the S_j are fixed given $\mathcal{G}$, with $\sum_j S_j = |E| \le s+2$, and each element of R lands in bucket j independently with probability p_j, so $N_j' \sim \text{Binomial}(|R|, p_j)$ with $|R| \le n$. Expanding $N_j^2 = N_j'^2 + 2S_jN_j' + S_j^2$:

- By Fact 2.5, $\sum_j \mathbb{E}[N_j'^2] \le \sum_j (np_j + n^2p_j^2) \le n + n^2 \cdot \frac{1+\chi^2}{n} = (2+\chi^2)\, n$, using $\sum_j p_j = 1$ and Lemma 6.6.
- $\sum_j 2S_j\mathbb{E}[N_j'] \le 2n\,(\max_j p_j) \sum_j S_j \le 2n(s+2)\big(\sum_j p_j^2\big)^{1/2} \le 2(s+2)\sqrt{(1+\chi^2)\, n}$.
- $\sum_j S_j^2 \le \big(\sum_j S_j\big)^2 \le (s+2)^2$. □

6.4 The main theorem

Theorem 6.8 (Expected linear time)**.** *Under Assumption (A), for $n > 64$,*

$$\mathbb{E}[T(n) \mid \mathcal{G}] \le (A + 10 + 5\chi^2)\, n + 10(s+2)\sqrt{(1+\chi^2)\, n} + 5(s+2)^2 + C.$$

With $A = 24$ and $s + 2 \le 1002$, this is $(34 + 5\chi^2)\, n + O(\sqrt{n})$. Averaging over $\mathcal{G}$ (and using $\mathbb{E}\sqrt{Y} \le \sqrt{\mathbb{E}Y}$),

$$\mathbb{E}[T(n)] \le \big(34 + 5\,\mathbb{E}[\chi^2]\big)\, n + 10(s+2)\sqrt{(1+\mathbb{E}[\chi^2])\, n} + 5(s+2)^2 + C,$$

so the expected time is linear whenever $\mathbb{E}[\chi^2]$ is bounded independently of n. Theorem 5.1 applies in every case.

Proof. By (1) and Lemma 4.2, $T \le W + 5\sum_j N_j^2$. By Lemma 4.1, $\mathbb{E}[W \mid \mathcal{G}] \le An + C$. Multiply the bound of Lemma 6.7 by 5 and add. □

Remark 6.9 (Why the total is linear). The proof charges every crowded bucket a *quadratic* cost, yet the total is still linear. The reason is Lemma 6.6: two random elements share a bucket with probability only about $(1+\chi^2)/n$, so crowded buckets are rare enough that even a quadratic cost adds up to $O(n)$. The constant $5(s+2)^2 \approx 5 \times 10^6$ comes from the at most 1002 sampled points possibly sharing buckets. It is a crude bound, but it does not grow with n.

6.5 When is χ^2 small?

Corollary 6.10 (Uniform data)**.** *If the data are uniform on an interval and the uniform model is chosen, then* $\chi^2 = 0$ *and* $\mathbb{E}[T(n) \mid \mathcal{G}] \le 34n + 10(s+2)\sqrt{n} + 5(s+2)^2 + C$.

Proof. The uniform model is $\widehat{F}(x) = (x - x_{\min})/(x_{\max} - x_{\min})$, so $\widehat{f} = 1/(x_{\max} - x_{\min})$ on $(x_{\min}, x_{\max})$. A uniform density restricted to $(x_{\min}, x_{\max})$ and rescaled is again uniform there, so $f_* = \widehat{f}$ and $\chi^2 = 0$. Apply Theorem 6.8. □

Corollary 6.11 (Empirical model)**.** *Suppose* f *is supported on a bounded interval* $[\alpha, \beta]$*, with* $0 < \mu \le f \le M$ *there, and consider the empirical model of step D3. Then*

$$\mathbb{E}\big[1 + \chi^2 \,\big|\, J, \iota, \kappa, x_{\min}, x_{\max}\big] \le \frac{2(s+1)}{s} \cdot \frac{M}{\mu} \le \frac{3M}{\mu}.$$

Hence the variant of the algorithm that always uses the empirical model satisfies $\mathbb{E}[T(n)] \le (29 + 15M/\mu)\, n + O(\sqrt{n})$.

Proof. Let $s' = |J \setminus \{\iota, \kappa\}| \ge s - 2$ be the number of sampled values strictly between $x_{\min}$ and $x_{\max}$. By the argument of Lemma 6.3(ii), given $(J, \iota, \kappa, x_{\min}, x_{\max})$ these s' values are independent with density f_*. Mapped through the CDF F_* of f_*, they become s' independent uniform points on $[0,1]$, while $x_{\min} \mapsto 0$ and $x_{\max} \mapsto 1$.

The sample points split $(x_{\min}, x_{\max})$ into $s'+1$ segments of positive length. (A repeated point, which occurs when J contains ι or κ, gives a segment of length zero, which does not matter.) On a segment of length ℓ_k, the empirical CDF rises by exactly $1/L$, so $\widehat{f} = 1/(L\ell_k)$. Let $\omega_k = \int_{\text{segment } k} f_*$. On $(x_{\min}, x_{\max})$ we have $\mu/Z \le f_* \le M/Z$, so $\omega_k \ge \mu\ell_k/Z$, that is $\ell_k \le Z\omega_k/\mu$, and $\int_{\text{segment } k} f_*^2 \le (M/Z)\,\omega_k$. Therefore

$$1 + \chi^2 = \sum_k L\ell_k \int_{\text{segment } k} f_*^2 \le \sum_k L \cdot \frac{Z\omega_k}{\mu} \cdot \frac{M}{Z}\,\omega_k = \frac{LM}{\mu} \sum_k \omega_k^2.$$

The ω_k are the gaps between s' independent uniform points on $[0,1]$ (including the gaps to 0 and to 1). Each of the $s'+1$ gaps has the $\mathrm{Beta}(1, s')$ distribution, with second moment $\frac{2}{(s'+1)(s'+2)}$, so $\mathbb{E}\sum_k \omega_k^2 = \frac{2}{s'+2}$. With $L = s+1$ and $s'+2 \ge s$, this gives the first bound. The second uses $s = \min(n, 1000) \ge 65$.

For the running time, apply Theorem 6.8 with $\mathbb{E}[\chi^2] \le 3M/\mu - 1$: $34 + 5(3M/\mu - 1) = 29 + 15M/\mu$. □

In the actual code, the empirical model is replaced by a named model only when the named model passes the Kolmogorov–Smirnov test, and in that case Theorem 6.8 applies with the named model's χ^2.

Remark 6.12 (Normal data with a fitted Normal model). For a Normal density f with standard deviation σ and a Normal model with $\hat{\sigma}$, the integral $\int f^2/\widehat{f}$ is a Gaussian integral whose exponent

has x^2-coefficient $-\frac{1}{\sigma^2}+\frac{1}{2\hat{\sigma}^2}$. So it is finite exactly when $\hat{\sigma}^2 > \sigma^2/2$. For small fitting errors, a second-order expansion (with the Fisher information as coefficients) gives $\chi^2(f \,\|\, \widehat{f}) \approx (\Delta\mu/\sigma)^2 + 2(\Delta\sigma/\sigma)^2$. With about 1000 sample points the relative errors are about 3% and 2%, so $\chi^2 \approx 0.002$. Restricting to $(x_{\min}, x_{\max})$ changes little: $1 + \chi^2(f_* \| \widehat{f}) \le (1 + \chi^2(f \| \widehat{f}))/Z^2$, and $Z \to 1$ as n grows.

7 Where linearity fails: repeated values

Proposition 7.1 (A repeated value forces $\Omega(n \log n)$)**.** *Suppose the values are independent and some value v has $\mathbb{P}(X = v) = \pi > 0$. Then for every $n \ge 130/\pi$, whatever model step D3 chooses,*

$$\mathbb{E}[T(n)] \ge \left(1 - e^{-\pi n/8}\right) \frac{\pi n}{2} \log \frac{\pi n}{2}.$$

Proof. Let K be the number of copies of v, so $K \sim \text{Binomial}(n, \pi)$. All copies have the same bucket $\beta(v)$, because β is a function of the value: no model can separate equal values. By the Chernoff bound, $\mathbb{P}(K < \pi n/2) \le e^{-\pi n/8}$. If $K \ge \pi n/2 \ge 65$, that bucket has more than 64 elements and is merge-sorted, which by Fact 2.2 costs at least $N(\lfloor \log N \rfloor + 1) \ge K \log K \ge \frac{\pi n}{2} \log \frac{\pi n}{2}$, where $N \ge K$ is the bucket's size and $x \log x$ is increasing. □

Remark 7.2 (The benchmarks, and the fix)*.* This is exactly the timestamp benchmark: 73% of the files share one modification time, so $\pi \approx 0.73$, and the sort was slower than plain merge sort. File sizes have many smaller repeated values. (A density does not exist for such data, so χ^2 is undefined.)

The proof also shows what to change. The expensive step is merge-sorting a bucket whose values are all equal. A single pass that checks whether all values in a bucket are equal costs $O(N)$ and makes such a bucket free to finish, which removes the $\Omega(n \log n)$ term for buckets holding one repeated value. Buckets that mix a repeated value with other values need a separate argument, like the pure/mixed bucket analysis in the earlier proof.

8 Discussion

Remark 8.1 (Why the $\Omega(n \log n)$ sorting lower bound does not apply)*.* Any *comparison* sort needs $\Omega(n \log n)$ comparisons, even on average over random inputs, because it must tell apart $n!$ orderings. Step D4 does not compare elements with each other: it computes $\left\lfloor n\widehat{F}(x) \right\rfloor$ with arithmetic, which locates x among n buckets in $O(1)$ time. All of the speed comes from this, and it pays off only when the model fits (χ^2 bounded).

Remark 8.2 (Why the sample size must be a constant)*.* The empirical CDF finds x's segment by binary search over the sample, which costs log(sample size) steps. With a fixed sample of 1000 this is at most 10 steps (Fact 2.3). If the sample grew with n, placement alone would cost $\Theta(n \log n)$. The price of a fixed sample is that the empirical model's accuracy, and so its χ^2, does not improve as n grows; Corollary 6.11 shows that it stays bounded anyway.

Remark 8.3 (Floating point)*.* The proofs assume exact arithmetic. In floating point, correctness depends only on Lemma 3.1, that is, on the computed $\widehat{F}$ being non-decreasing. This holds because IEEE subtraction and division by a positive number are monotone, `erf` and `exp` are monotone in standard maths libraries, and the empirical CDF's interpolation fraction stays in $[0, 1]$ after rounding. It also needs differences such as $x_{\max} - x_{\min}$ not to overflow, which holds for values below about 10^{308} in size. The running-time bounds do not depend on rounding.

Remark 8.4 (Connection to the measurements). For uniform and Normal data the fitted models have $\chi^2 \approx 0$, and the measured time per element stayed roughly flat from 10^4 to 10^6 elements, as Theorem 6.8 predicts. The extra slowdown observed above 10^5 elements also hit merge sort and Python's built-in sort, so it comes from the machine rather than the algorithm (part of it was measured to be the garbage collector; the rest is most likely memory-cache misses), and the RAM model does not capture it.

A Where every number comes from

Number	Origin
64	`BIG_BUCKET` in the code: the largest bucket that is insertion-sorted.
1000	`SAMPLE_SIZE` in the code. With $x_{\min}$ and $x_{\max}$ added, the model sees at most 1002 points.
1.36	KS acceptance threshold factor: the 95% point of the Kolmogorov distribution. It affects which model is chosen, not the bounds.
10	Binary-search iterations on ≤ 1002 points: $\lfloor \log 1002 \rfloor + 1 = 10$ (Fact 2.3).
5	Steps per element per merge-sort level: copy, slice, two for merging, call overhead (Fact 2.2). Also $\text{cost}(k) \leq 5k^2$ (Lemma 4.2).
$5n$	Expected steps of `random.sample` (Fact 2.4).
24 (A)	Sum of the per-element costs in Lemma 4.1.
74	64 (small buckets) $+ 5 \cdot 2$ (from $\lceil \log k \rceil + 1 \leq \log n + 2$) in Theorem 5.1.
98	$A + 74$.
10, 34	$5 \cdot 2$ from $\mathbb{E}[N_j^2] < 2$; and $A + 10$ (Corollary 6.2, Theorem 6.8).
$2 + \chi^2$	$\sum_j \mathbb{E}[N_j'^2] \leq n + n^2 \cdot \frac{1+\chi^2}{n}$ (Lemma 6.7).
$s + 2$	At most this many elements (the sample, $x_{\min}$, $x_{\max}$) are not independent of the model (Lemma 6.3).
$2/(s' + 2)$	Expected sum of squared gaps between s' uniform points (Corollary 6.11).
$3M/\mu$	$2(s+1)/s \cdot M/\mu$ with $s \geq 65$ (Corollary 6.11).
29, 15	$34 + 5(3M/\mu - 1)$ (Corollary 6.11).
$130/\pi$, 8	Need $\pi n/2 \geq 65$ for the safety net to trigger; Chernoff bound $\mathbb{P}(K < \frac{1}{2}\mathbb{E}K) \leq e^{-\mathbb{E}K/8}$ (Proposition 7.1).